MAIO

Rethinking Zero-Copy Networking

Markuze Alex

VMware

User-space Networking

For fast networks > 1 Gb/s

The Choice:

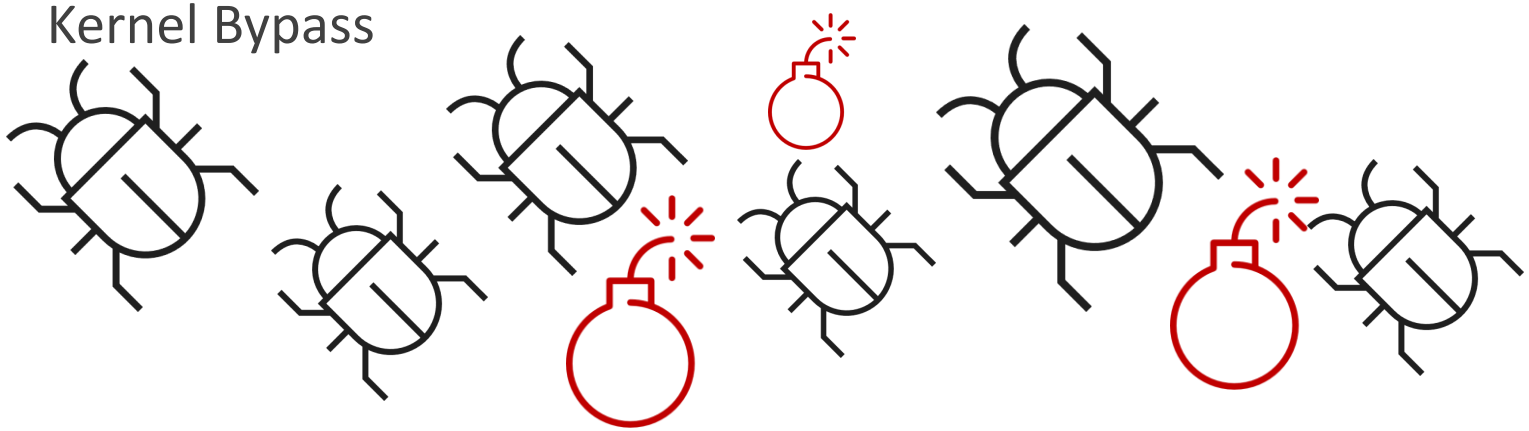
BSD Sockets



```
import socket                                                  /* Import */
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)           /* Create */
s.connect((destination, port))                                 /* Connect - Routing */
s.sendall('Hello, world’)                                      /* Send     - TCP/IP  */
```



Kernel Bypass



Zero-Copy Network I/O

The Holy Grail

Why Zero-Copy?

1. Performance

   70% CPU spent on copy operations

2. Performance

   Cache line thrashing

3. Performance

   Memory Bandwidth

Zero-Copy Network I/O

Why copy?

The Problem:

Network packet arrival order is random!

1. Kernel provides memory for I/O
2. Packet Arrives
3. Data *copied* to correct process.

Storage\GPU\etc… data arrival order is known.

1. Process asks for I/O
2. User provides buffer
3. Data arrives

Zero-Copy Network I/O

Challenges

Challenges posed:

1. Isolation — Kernel memory from user
2. Isolation — Packet headers from user
3. Isolation — Different users

Zero-Copy Approaches

40 years of innovations

1. Dynamic Remapping
   - MSG_ZERO_COPY, TCP_MMAP, COW (FreeBSD)
2. Kernel Bypass
   - DPDK, NetMap, AF_XDP
3. Limited use-case
   - splice, sendfile, SOCKMAP
4. Specialized HW
   - RDMA – iWARP, Infiniband, ROCE
5. Shared Memory
   - No

Zero-Copy with Shared Memory

MAIO - Memory allocation for I/O

The Solution: Memory Segmentation - I/O (only) Pages*

1. MAIO pages used for I/O
2. Packet arrives
3. Data page delivered to process

How does this help?!

1. Isolation - Kernel Memory: No Kernel data is ever located on an I/O page
   - All data is by/for the user
2. Isolation - between processes: HW supports is needed (resolves the randomness).
   - Intel ADQ
   - NVIDIA QP (DPDK Bifurcated driver)
   - Virtualization paravirt-drivers (virtio, netvsc, vmxnet)
3. Isolation - control path: Copy *only* network headers

Zero-Copy with Shared Memory

Where is the catch?

Limitations:

Zero-Copy semantics vs Copy Semantics

1. Buffers change ownership.
    - On TX the buffer is owned by Kernel
2. No serialisation/linearisation.
    - On RX the buffer size is limited by MTU*

MAIO in practice

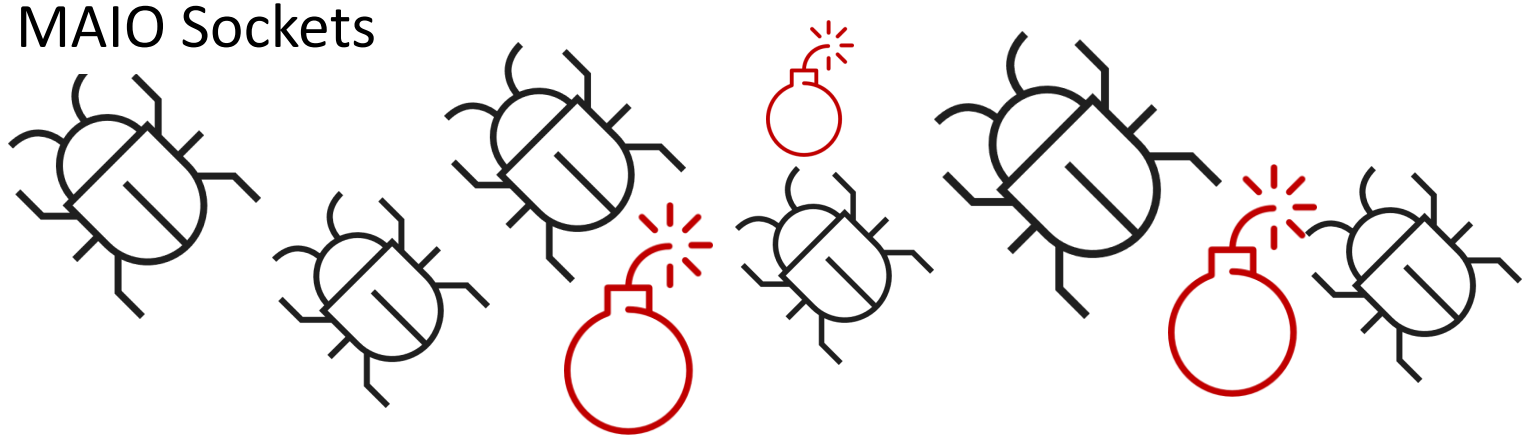
For fast networks > 1 Gb/s

The Choice:

BSD Sockets

```
import socket                                                  /* Import */
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)           /* Create */
s.connect((destination, port))                                 /* Connect - Routing */
s.sendall('Hello, world’)                                      /* Send     - TCP/IP  */
```

MAIO Sockets



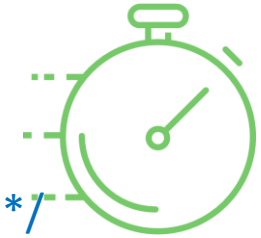
MAIO in practice

For fast networks > 1 Gb/s

The Choice:

BSD Sockets

```
import socket                                          /* Import */
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)   /* Create */
s.connect((destination, port))                         /* Connect - Routing */
s.sendall('Hello, world')                              /* Send     - TCP/IP  */
```



MAIO Sockets

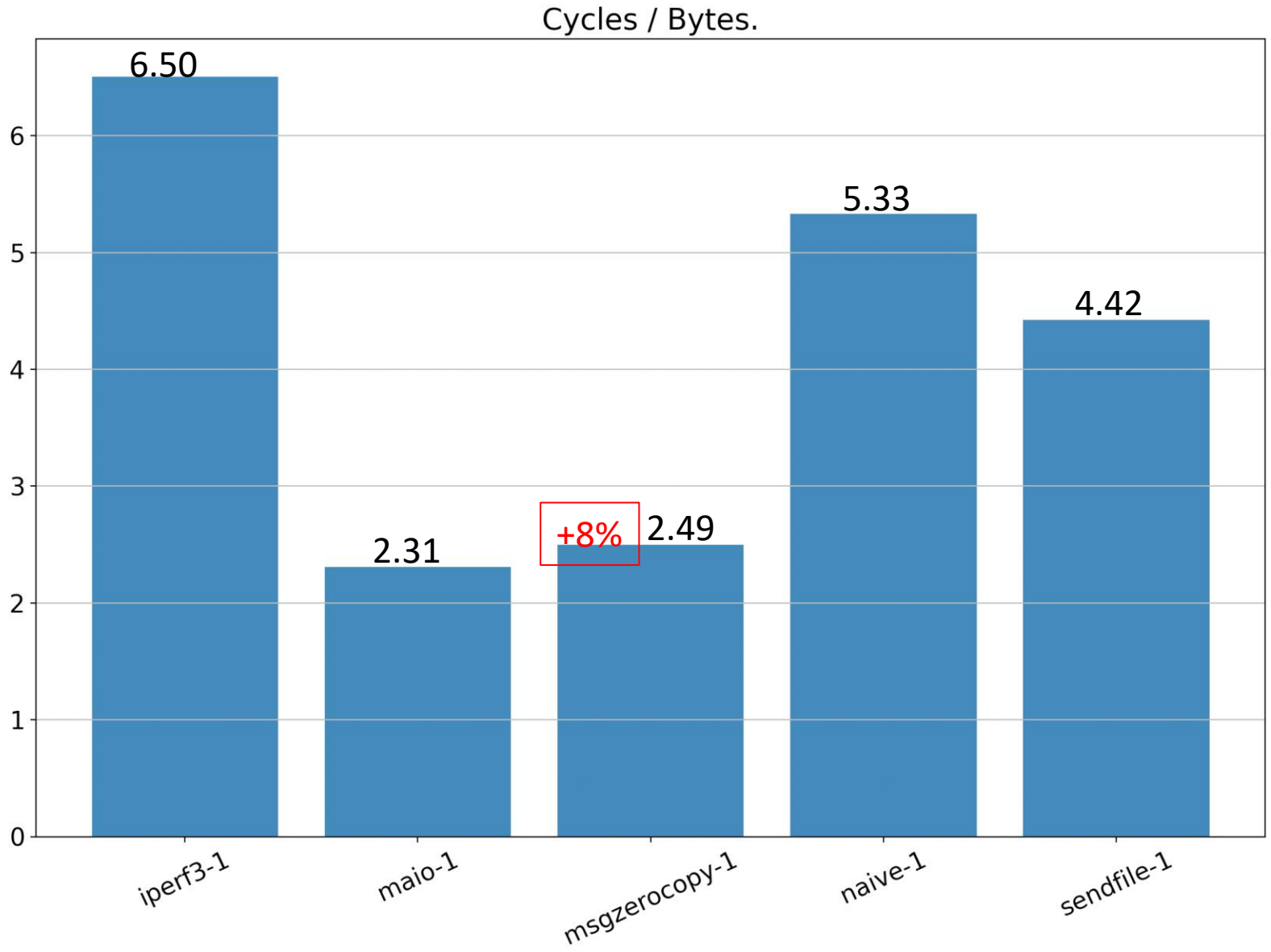
```
void *cache    = init_hp_memory(PAGE_CNT);          /*Init MAIO pages          */
int   idx      = create_connected_socket(dip, port); /* Create a TCP socket     */
char *buffer   = alloc_page(cache);                 /* get a 4K buffer from HP */

init_tcp_ring(idx, cache);                          /* init the Async TX ring  */
send_buffer(idx, buffer, len, flags);
…
get_buffer_state(buffer);                           /* get the page/chunk state*/
```

Comparing TX loop of 16KB buffers

1 Thread

- 16 Core VM on GCP
- 32Gb/s Egress
- 64 GB Memory
- 16K * 2K buffers with 1 Thread
- Cycles/Byte – *Lower is Better*



Comparing TX loop of 16KB buffers

1 Thread

- 16 Core VM on GCP
- 32Gb/s Egress
- 64 GB Memory
- 16K * 2K buffers with 1 Thread
- Cycles/Byte – *Lower is Better*

MAIO Send

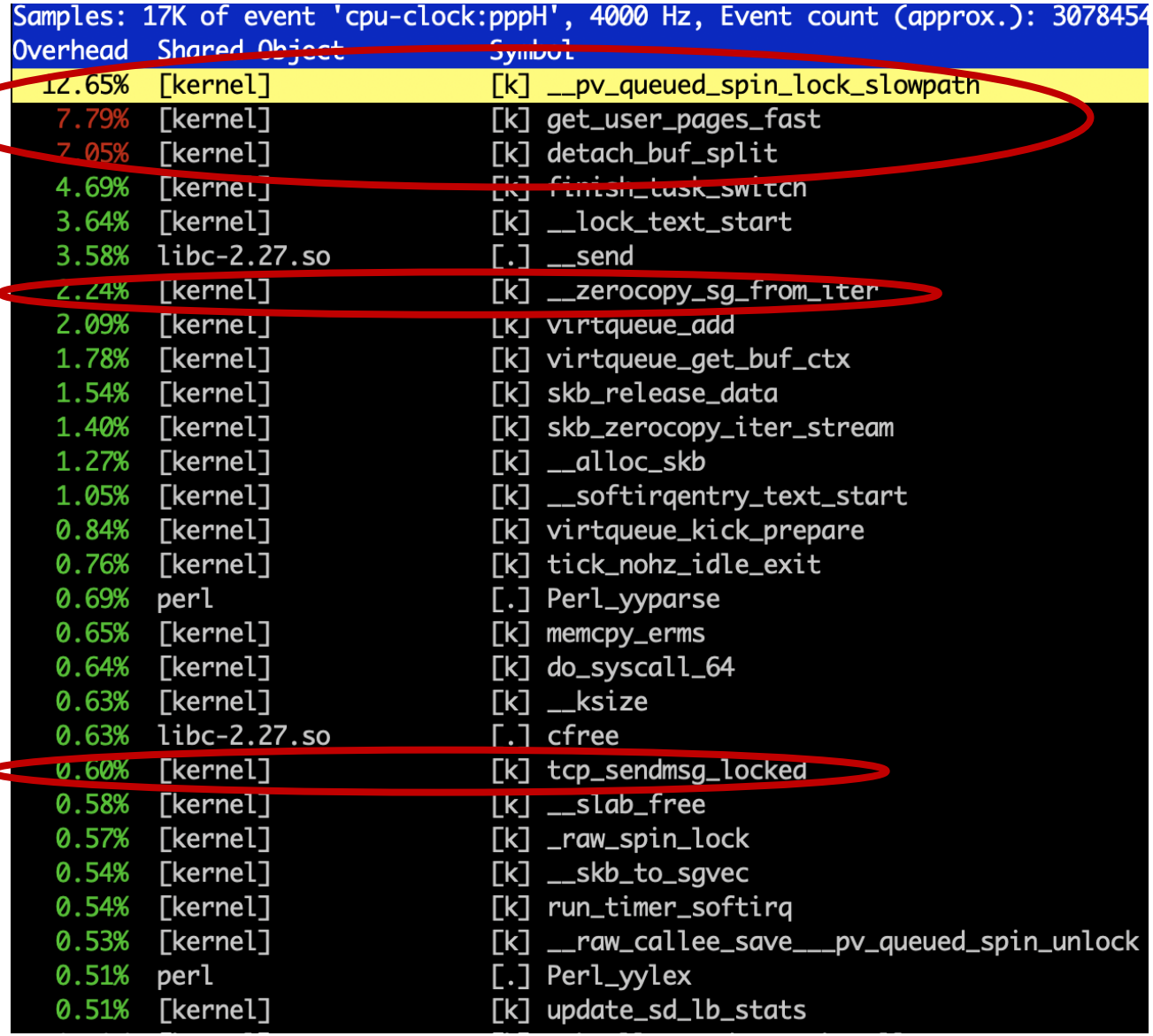
```
Samples: 9K of event 'cpu-clock:pppH', 4000 Hz, Event count (approx.): 1677661402
Overhead  Shared Object          Symbol
  11.27%  [kernel]               [k] do_tcp_sendpages
   9.78%  [kernel]               [k] maio_post_tx_tcp_page
   8.95%  [kernel]               [k] _raw_spin_lock_bh
   7.09%  [kernel]               [k] __raw_callee_save___pv_queued_spin_unlock
   3.73%  [kernel]               [k] _raw_spin_unlock_irqrestore
   3.31%  [kernel]               [k] detach_buf_split
   3.07%  [kernel]               [k] finish_task_switch
   2.26%  [kernel]               [k] __pv_queued_spin_lock_slowpath
   1.69%  [kernel]               [k] tcp_current_mss
   1.65%  [kernel]               [k] __alloc_skb
   1.54%  [kernel]               [k] skb_release_data
   1.46%  [kernel]               [k] virtqueue_add
   1.40%  [kernel]               [k] __local_bh_enable_ip
   1.31%  [kernel]               [k] tcp_established_options
   1.29%  libc-2.27.so           [.] write
   1.24%  [kernel]               [k] tcp_send_mss
   1.21%  [kernel]               [k] virtqueue_get_buf_ctx
   1.17%  [kernel]               [k] ipv4_mtu
   1.14%  [kernel]               [k] tcp_sendpage
   1.09%  [kernel]               [k] lock_sock_nested
   1.05%  [kernel]               [k] tick_nohz_idle_exit
   1.05%  [kernel]               [k] tcp_sendpage_locked
   1.04%  [kernel]               [k] release_sock
   0.99%  [kernel]               [k] __do_softirq
   0.75%  [kernel]               [k] tcp_release_cb
   0.74%  [kernel]               [k] __slab_alloc
   0.73%  maio_tcp_tx_client     [.] send_buffer
   0.61%  [kernel]               [k] tcp_tx_timestamp
   0.61%  [kernel]               [k] __ksize
   0.59%  [kernel]               [k] run_timer_softirq
   0.59%  [kernel]               [k] virtqueue_kick_prepare
   0.59%  perf                   [.] __symbols__insert
   0.59%  [kernel]               [k] __kmalloc_node_track_caller
   0.52%  [kernel]               [k] tcp_ack
   0.50%  [kernel]               [k] tcp_push
```

Comparing TX loop of 16KB buffers

1 Thread

- 16 Core VM on GCP
- 32Gb/s Egress
- 64 GB Memory
- 16K * 2K buffers with 1 Thread
- Cycles/Byte – *Lower is Better*

MSG_ZEROCOPY



Comparing TX loop of 16KB buffers

1 Thread

- 16 Core VM on GCP
- 32Gb/s Egress
- 64 GB Memory
- 16K * 2K buffers with 1 Thread
- Cycles/Byte – *Lower is Better*

```
Samples: 51K of event 'cpu-clock:pppH', 4000 Hz, Event count (approx.): 819276
Overhead  Shared Object       Symbol
  57.32%  [kernel]            [k] copy_user_enhanced_fast_string
   6.65%  [kernel]            [k] tcp_sendmsg_locked
   3.78%  [kernel]            [k] detach_buf_split
   1.33%  [kernel]            [k] __lock_text_start
   1.18%  [kernel]            [k] virtqueue_get_buf_ctx
   1.13%  [kernel]            [k] virtqueue_kick_prepare
   1.05%  libc-2.27.so        [.] __send
   1.04%  [kernel]            [k] iov_iter_advance
   0.92%  [kernel]            [k] _raw_spin_lock
   0.89%  [kernel]            [k] get_page_from_freelist
   0.86%  [kernel]            [k] __softirqentry_text_start
   0.75%  [kernel]            [k] __free_pages_ok
   0.71%  [kernel]            [k] __ip_queue_xmit
   0.58%  [kernel]            [k] skb_release_data
   0.53%  [kernel]            [k] copyin
   0.51%  [kernel]            [k] skb_release_head_state
```

Simple Socket

Comparing TX loop of 16KB buffers

1 Thread

- 16 Core VM on GCP
- 32Gb/s Egress
- 64 GB Memory
- 16K * 2K buffers with 1 Thread
- Cycles/Byte – *Lower is Better*

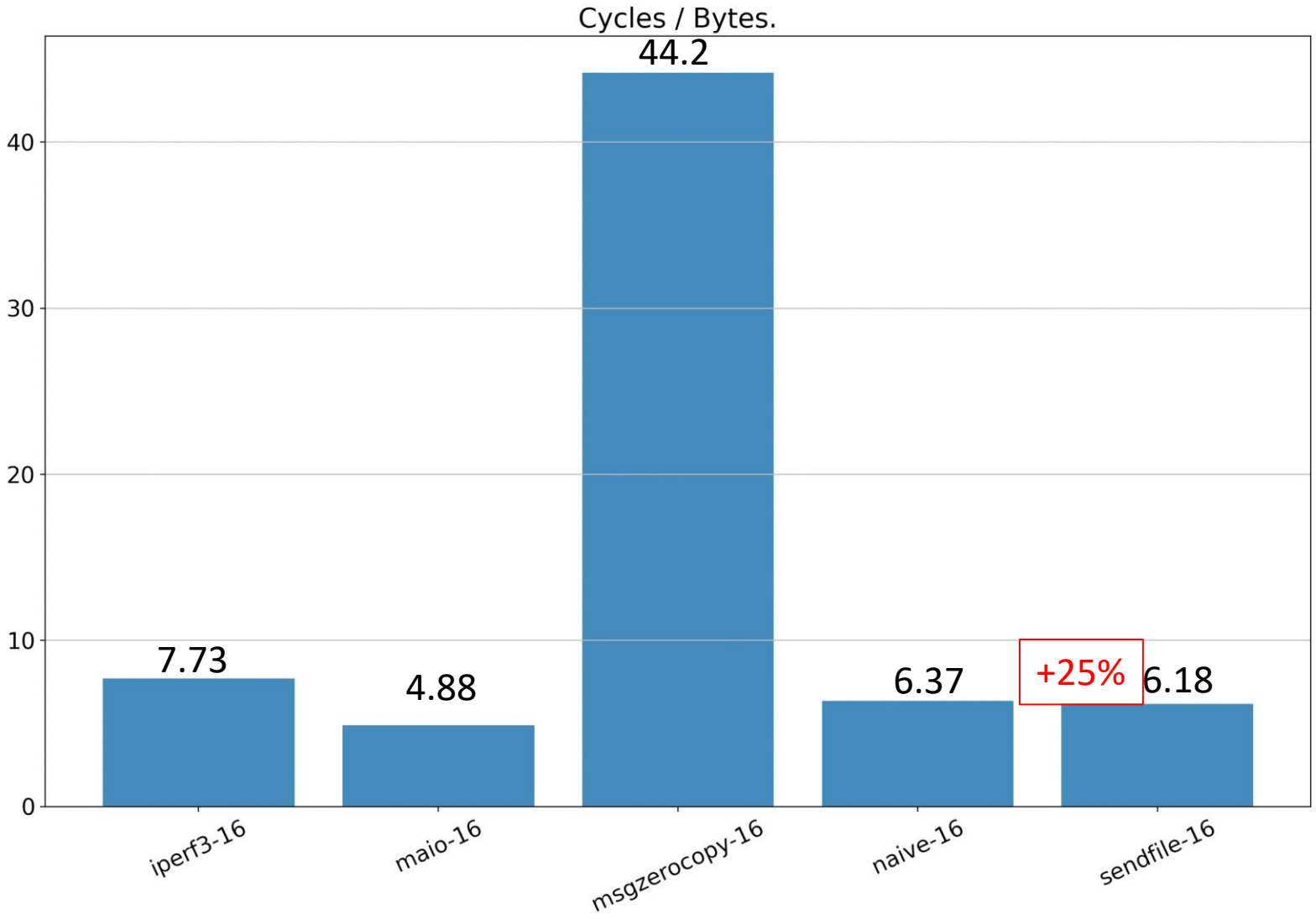
Sendfile

```
Samples: 381K of event 'cpu-clock:pppH', 4000 Hz, Event count (approx.): 110466710
Overhead  Shared Object          Symbol
   9.29%  libc-2.27.so           [.] sendfile
   8.45%  libc-2.27.so           [.] __lseek
   5.30%  [kernel]               [k] find_get_entry
   3.99%  [kernel]               [k] __pv_queued_spin_lock_slowpath
   3.50%  [kernel]               [k] do_syscall_64
   3.27%  [kernel]               [k] detach_buf_split
   2.88%  [kernel]               [k] do_tcp_sendpages
   2.46%  [kernel]               [k] _raw_spin_lock_bh
   1.75%  [kernel]               [k] shmem_file_read_iter
   1.71%  [kernel]               [k] __raw_callee_save___pv_queued_spin_unlock
   1.64%  [kernel]               [k] generic_file_splice_read
   1.55%  [kernel]               [k] __fget_light
   1.38%  [kernel]               [k] rw_verify_area
   1.38%  [kernel]               [k] _raw_spin_unlock_irqrestore
   1.25%  [kernel]               [k] fsnotify
   1.20%  [kernel]               [k] copy_page_to_iter
   1.13%  [kernel]               [k] xas_start
   1.11%  [kernel]               [k] __do_softirq
   1.04%  [kernel]               [k] virtqueue_get_buf_ctx
   1.02%  [kernel]               [k] xas_load
   0.94%  [kernel]               [k] virtqueue_add
   0.93%  [kernel]               [k] page_mapping
   0.93%  [kernel]               [k] page_cache_pipe_buf_release
   0.92%  [kernel]               [k] set_page_dirty
   0.86%  [kernel]               [k] inet_sendpage
   0.83%  [kernel]               [k] common_file_perm
   0.83%  [kernel]               [k] __fsnotify_parent
   0.82%  [kernel]               [k] splice_direct_to_actor
   0.82%  [kernel]               [k] tcp_sendpage
   0.76%  [kernel]               [k] shmem_getpage_gfp
   0.75%  [kernel]               [k] do_sendfile
   0.74%  [kernel]               [k] find_lock_entry
   0.74%  [kernel]               [k] finish_task_switch
   0.71%  [kernel]               [k] skb_release_data
   0.69%  [kernel]               [k] security_file_permission
   0.66%  [kernel]               [k] __splice_from_pipe
   0.61%  [kernel]               [k] unlock_page
```

Comparing TX loop of 16KB buffers

16 Threads

- 16 Core VM on GCP
- 32Gb/s Egress
- 64 GB Memory
- 16K * 2K buffers with 16 Threads
- Cycles/Byte – *Lower is Better*



Kernel Memory Segmentation

What is it?

- Memory Intended for a specific use case.
- Allocate with a dedicated allocator.
- Mange and Free with standard put/page operations.

include/linux/mm_types.h

- struct page {} (64B metadata)
- get/put_page()
- compound_head()
- ~~page_ref_inc~~

Kernel Memory Segmentation

What is it?

- Memory Intended for a specific use case.
- Allocate with dedicated allocator.
- Mange and Free with standard put/page operations.

Example DEV_PAGEMAP

```
static inline void put_page(struct page *page)
{
        page = compound_head(page);

    if (put_devmap_managed_page(page))
            return;

        if (put_page_testzero(page))
                __put_page(page);
}
```

HugePages and Kernel Memory Segmentation

- MAIO uses 2MB HP (compound pages).
- First TailPage holds uaddr for fast memory translation and is_maio_page

```
struct {        /* Tail pages of compound page */
                  unsigned long compound_head;    /* Bit zero is set */

                  /* First tail page only */
                  unsigned char   compound_dtor;
                  unsigned char   compound_order;
                  u16             elem_order;             /* Plug a 2B Hole*/
                  atomic_t        compound_mapcount;
                  unsigned int    compound_nr;            /* 1 << compound_order */
                  /* 4B Hole*/
                  unsigned long uaddr;          /* Uaddr*/
    };
```

HugePages and MAIO

```
void *cache        = init_hp_memory(PAGE_CNT);                    /*Init MAIO pages          */
```

1. Get PAGE_CNT of 2MB HP
2. Kernel: Create a Memory Translation Table.
   8B entry * 1 HP (2MB)
3. Kernel: Set page[1]->uaddr
4. Init custom memory management. (linked list of 4K and 16K)

- Head Page is not used for I/O as ref count never 0.
- Tail Pages have RC = 0 while in user space.
- get/put_page operate on Tail Page not the Head Page.

Socket Managment

```
int idx   = create_connected_socket(dip, port);                    /* Create a TCP socket        */
```

1. Kernel: Create a Kernel TCP socket.
2. Kernel: Connect to destination.
3. Returns internal identifier for the socket.

Async TX Ring

```
init_tcp_ring(idx, cache);                    /* init the Async TX ring      */
```

1. Allocate and Init a ring for async TX. Uses cache.
2. Kernel: affiliate socket and async ring. Uses idx.
3. Kernel: Create a kernel thread

Zero-Copy TCP TX

send_buffer(idx, buffer, len, flags);

1. Post a MAIO buffer on the ring[idx]
2. Wake-Up the kernel Thread (unless flags have MORE set)

Async TX Thread

1. Poll the ring[idx]
2. Use MTT to get kernel address
3. Send the buffer with tcp_sendpage

Page State TX

Who owns a sent buffer?

- RC handles correct state. get_page on each TX.
- Options.
  - Socket notification (sock_zerocopy_callback/ubuf_info)
  - Opportunistic state management on Head Page
    - Transition 0 – 1.
  - Sys-call to query page_ref_cnt

Head Page – shadow map of 4KB pages.
512 4K pages in 2MB
8B for each 4K Page on Head Page

Zero-Copy TCP RX

tcp_receive_sg(idx, io_vec, cache)

- Collect a sg list from the TCP socket
- kernel: Translate kernel address to user address using page->uaddr
- Kernel: refill kernel pages

Kernel Modifications

RX:

- *skb_zerocopy_iter_stream* modification skb_zerocopy_sg_from_iter (tcp_mmap)

Init:

- A MAIO implemenmtation: MAIO allocator for RX ring
  - Magazine allocator
- Driver modification (AF_XDP)

Page State RX

Who owns the received buffer?

- RC handles correct state.
- Options.
  - Opportunistic state management on Head Page
  - One RX per page (XDP)
    - RX buffer ownership transferred on RX

Who owns the mixed-use page?

DPDK MAIO-PMD

Why?

- HW Agnostic
- Standard Linux Tools
- Hybrid use-case
  - Decrypt in user-space and plain text via network stack

How?

- RX ring
- Direct xmit (pktgen)
- Already HP and own memory mangment

Next Steps

1. Upstreaming
    a. Linux Kernel
    b. DPDK
    c. User Library
2. Dethrone SRIOV
    a. QEMU is just another process
    b. Nested MAIO support
3. Efficient Copy Semantics with Intel SIMD - SSE (?)
    a. Vector operations are fast
    b. 6 memory channels

Github

Kernel & User Lib: git@github.com:Markuze/maio_rfc.git

DPDK: git@github.com:Markuze/maio_dpdk_rfc.git

Email : amarkuze@vmware.com